

Data Structures: The Basics

A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently. Choosing the right data structure is often the difference between a fast program and a slow one.

Arrays

An array stores elements in contiguous memory, one after another. Each element is reached by its index in constant time, $O(1)$. Because the size is usually fixed when the array is created, inserting or deleting in the middle requires shifting the other elements, which costs $O(n)$ time.

Linked Lists

A linked list stores each element in a node that also holds a pointer to the next node. Unlike an array, the nodes are not contiguous in memory. Inserting or deleting at a known position is $O(1)$ because only a couple of pointers change, but finding an element requires walking the list from the start, which is $O(n)$.

Stacks

A stack is a Last-In, First-Out (LIFO) structure: the last item pushed on is the first item popped off. Think of a stack of plates. Stacks are used for function call management, undo features, and depth-first traversal. Push and pop are both $O(1)$.

Queues

A queue is a First-In, First-Out (FIFO) structure: items are added at the back and removed from the front, like people waiting in line. Queues are used for scheduling and breadth-first traversal. Enqueue and dequeue are both $O(1)$.

Hash Tables

A hash table maps keys to values using a hash function that turns a key into an index in an underlying array. On average, lookups, insertions, and deletions are $O(1)$. When two keys hash to the same index ? a collision ? the table resolves it with techniques such as chaining (a linked list per bucket) or open addressing. In the worst case, when many keys collide, operations can degrade to $O(n)$.

Trees

A tree is a hierarchical structure of nodes with a single root and no cycles. In a binary search tree (BST), every node's left subtree holds smaller keys and its right subtree holds larger keys, which makes searching, inserting, and deleting take $O(\log n)$ time when the tree is balanced. If the tree becomes unbalanced (essentially a long chain), those operations degrade to $O(n)$, which is why self-balancing trees such as AVL and red-black trees exist.

Big-O Notation

Big-O notation describes how an algorithm's running time or memory grows as the input size n grows. $O(1)$ is constant time, $O(\log n)$ is logarithmic, $O(n)$ is linear, and $O(n^2)$ is quadratic. It expresses the worst-case upper bound and ignores constant factors, so it captures how an algorithm scales rather than its exact speed on a particular machine.